李纲
宁波大学
信息科学与工程学院

线性规划与网络流

算法分析与设计

Analysis and Design of Computer Algorithms

- **学习要点**
- **理解线性规划算法模型**
- **掌握解线性规划问题的单纯形算法**
- **理解网络与网络流的基本概念**
- **掌握网络最大流的增广路算法**
- **掌握网络最大流的预流推进算法**
- **掌握网络最小费用流的消圈算法**
- **掌握网络最小费用流的最小费用路算法**
- **掌握网络最小费用流的网络单纯形算法**

- **什么是线性规划**

- **假如你要竞选市长。要当上市长，你必须有5万的城市居民的投票、10万郊区居民的投票以及2.5万农村居民的投票。**
- **你有以下四种方案使你获得更多的投票：**
- **1.建设道路**
- **2.加强枪支管制**
- **3.发放农业津贴**
- **4.减免油税**

$-2x_1+8x_2+0x_3+10x_4 >= 50$（城市居民）

$5x_1+2x_2+0x_3+0x_4 >= 100$（郊区居民）

$3x_1-5x_2+10x_3-2x_4 >= 25$（农村居民）

$x_1, x_2, x_3, x_4 >= 0$（开支不可能为负）

	城市	郊区	农村
建设道路	-2	5	3
枪支管制	8	2	-5
农业津贴	0	0	10
减免油税	10	0	-2

- **比如第一行第一列-2代表在建设道路上每增加1万元的支出，会减少2千人的城市居民选票；第一行第二列5表示在建设道路上每增加1万元支出，会增加5千人的郊区居民选票；第二行第三列表示在枪支管制上每增加1万元支出，会减少5千人的农村居民选票。**
- **你要用最少的支出来获得足够的选票当上市长，假设初始时，你的投票数都为0。**

- 假设4种项目的支出分别为x_1、x_2、x_3、x_4万元，
- 目标：最小化$x_1+x_2+x_3+x_4$（总支出最小）
- 限制：
- $-2x_1+8x_2+0x_3+10x_4 >= 50$（城市居民）
- $5x_1+2x_2+0x_3+0x_4 >= 100$（郊区居民）
- $3x_1-5x_2+10x_3-2x_4 >= 25$（农村居民）
- $x_1, x_2, x_3, x_4 >= 0$（开支不可能为负）

- **线性规划：在满足一些线性等式或者不等式的条件下，最优化一个线性函数。**

最大化$c_1x_1+c_2x_2+c_3x_3+c_4x_4$

满足:

$a_{11}x_1+a_{21}x_2+a_{31}x_3+a_{41}x_4<=b_1$

$a_{12}x_1+a_{22}x_2+a_{32}x_3+a_{42}x_4<=b_2$

$a_{13}x_1+a_{23}x_2+a_{33}x_3+a_{43}x_4<=b_3$

$x_1, x_2, x_3, x_4>=0$

- **资源配置类型**
- 有m种资源和n个项目，每个资源都是有限的，设它们的上限为b_j(1 <= j <= m)。假设第i个项目做出x_i的成果量，可以获得c_i*x_i的收益，同时会消耗第j种资源$a_{ij}*x_i$。求最大收益。
- 比如下面的这个问题就是一个资源优化配置问题：
- 某工厂现在分别有钢材、木材、塑料b_1、b_2、b_3吨，工厂可以生产4种产品，第i种产品每生产一吨可以获得c_i万的收益，但是要耗费a_{i1}吨钢材，a_{i2}吨木材以及a_{i3}吨塑料。求工厂的最大收益。

- 假设4种产品产量分别为x_1、x_2、x_3、x_4吨
- 最大化$c_1x_1+c_2x_2+c_3x_3+c_4x_4$
- 满足:
- $a_{11}x_1+a_{21}x_2+a_{31}x_3+a_{41}x_4 <= b_1$
- $a_{12}x_1+a_{22}x_2+a_{32}x_3+a_{42}x_4 <= b_2$
- $a_{13}x_1+a_{23}x_2+a_{33}x_3+a_{43}x_4 <= b_3$
- $x_1, x_2, x_3, x_4 >= 0$

最小化$c_1x_1+c_2x_2+c_3x_3$

满足：

$a_{11}x_1+a_{21}x_2+a_{31}x_3 >= b_1$

$a_{12}x_1+a_{22}x_2+a_{32}x_3 >= b_2$

$a_{13}x_1+a_{23}x_2+a_{33}x_3 >= b_3$

$a_{14}x_1+a_{24}x_2+a_{34}x_3 >= b_4$

$x_1, x_2, x_3, x_4 >= 0$

- **最佳物资供给类型**

- 有m种需求要满足，假设第j种需求至少要b_j的量才能满足。现在有n种物资，其中每单位第i种物资会提供给第j种需求a_{ij}的量。假设第i种物资供给了x_i单位，要支付费用c_i*x_i 。在满足所有需求的前提下，使总费用最小。
- 下面的问题就是一个最佳物资供给问题：
- 一个人一天至少要摄入b_1克糖类、b_2克脂肪、b_3克蛋白质以及b_4克维生素。米饭的价格为每克c_1元，每克米饭会提供a_{11}克糖类、a_{12}克脂肪、a_{13}克蛋白质以及a_{14}克的维生素；蔬菜每克c_2元，每克蔬菜回提供a_{21}克糖类、a_{22}克脂肪、a_{23}克蛋白质以及a_{24}克维生素；肉类每克c_3元，每克肉类提供a_{31}克糖类、a_{32}克脂肪、a_{33}克蛋白质以及a_{34}克维生素。请问至少要多少钱才能满足人一天的营养需求?

- 上述问题可以描述为下面的线性规划问题：
- 假设人一天吃米饭x_1克、蔬菜x_2克、肉类x_3克
- 最小化$c_1x_1+c_2x_2+c_3x_3$
- 满足：
- $a_{11}x_1+a_{21}x_2+a_{31}x_3 >= b_1$
- $a_{12}x_1+a_{22}x_2+a_{32}x_3 >= b_2$
- $a_{13}x_1+a_{23}x_2+a_{33}x_3 >= b_3$
- $a_{14}x_1+a_{24}x_2+a_{34}x_3 >= b_4$
- $x_1, x_2, x_3, x_4 >= 0$

- **多物流网络类型**

- 多物网络流基本上和一般的网络流一致，唯一的区别就是多物网络流有k个源点和汇点，k可能大于1。假设第i个源点为si，第i个汇点为ti （1 <= i <= k）。多物网络流的问题就是要求一个满足si到ti的流量都为fi的可行流。

- 在一个地图上，某铁路公司有4项目。第i个项目要建造一条从城市si到ti的铁路。现在有一些铁路段可以供公司选择建造，每一段都是从一个城市到另一个城市的，且每一段铁路的造价是已知的。由于项目之间是独立的，每一段铁路只能被一个项目所拥有、建造，造好以后也只能被此项目所使用。求完成4个项目的最小费用。

8.1 线性规划问题和单纯形算法

- **线性规划问题及其表示**
- **线性规划问题可表示为如下形式：**

$$\max \quad \sum_{j=1}^{n} c_j x_j \tag{8.1}$$

$$\text{s.t.} \quad \sum_{t=1}^{n} a_{it} x_t \le b_i \qquad i = 1,2,\cdots,m_1 \tag{8.2}$$

$$\sum_{t=1}^{n} a_{jt} x_t = b_j \qquad j = m_1+1,\cdots,m_1+m_2 \tag{8.3}$$

$$\sum_{t=1}^{n} a_{kt} x_t \ge b_k \qquad k = m_1+m_2+1,\cdots,m_1+m_2+m_3 \tag{8.4}$$

$$x_t \ge 0 \qquad t = 1,2,\cdots,n \tag{8.5}$$

- 变量满足约束条件(8.2)-(8.5)式的一组值称为线性规划问题的一个**可行解**。
- 所有可行解构成的集合称为线性规划问题的**可行区域**。
- 使目标函数取得极值的可行解称为**最优解**。
- 在最优解处目标函数的值称为**最优值**。

- 有些情况下可能不存在最优解。
- 通常有两种情况：
- (1)根本没有可行解，即给定的约束条件之间是相互排斥的，**可行区域为空集**；
- (2)目标函数没有极值，也就是说在n维空间中的某个方向上，**目标函数值可以无限增大**，而仍满足约束条件，此时**目标函数值无界**。

$$\max \quad z = x_1 + x_2 + 3x_3 - x_4$$

$$\begin{aligned} x_1 + 2x_3 &\le 18 \\ 2x_2 - 7x_4 &\le 0 \\ x_1 + x_2 + x_3 + x_4 &= 9 \\ x_2 - x_3 + 2x_4 &\ge 1 \\ x_i \ge 0 \qquad & i = 1,2,3,4 \end{aligned}$$

- **这个问题的解为 (x1,x2,x3,x4) = (0,3.5,4.5,1)；最优值为16。**
- **n=4; m1=2;m2=m3=1; m=m1+m2+m3=4;**

具体做法是，引入松弛/剩余变量，利用松弛变量的非负性，将不等式转化为等式。
松驰变量记为y_i，共有m_1+m_3个。
在求解过程中，应当将松弛变量与原来变量同样对待。求解结束后，抛弃松弛变量。
注意松弛变量前的符号由相应的原不等式的方向所确定。

$$\begin{aligned} x_1+2x_3 &\le 18 \\ 2x_2-7x_4 &\le 0 \\ x_1+x_2+x_3+x_4 &= 9 \\ x_2-x_3+2x_4 &\ge 1 \end{aligned} \qquad \begin{aligned} x_1+2x_3+y_1 &= 18 \\ 2x_2-7x_4+y_2 &= 0 \\ x_1+x_2+x_3+x_4 &= 9 \\ x_2-x_3+2x_4-y_3 &= 1 \end{aligned}$$

线性规划基本定理

- 约束条件(8.2)-(8.5)中*m*个约束以等号满足的可行解称为线性规划问题的基本可行解。
- 若n>m，则基本可行解中至少有n-m个分量为0，也就是说，基本可行解中最多有m个分量非零。
- **线性规划基本定理**：如果线性规划问题有最优解，则必有一基本可行最优解。
- 上述定理的重要意义在于，它把一个最优化问题转化为一个组合问题，即在(8.2) -(8.5)式的m+n个约束条件中，确定最优解应满足其中哪n个约束条件的问题。
- 由此可知，只要对各种不同的组合进行测试，并比较每种情况下的目标函数值，直到找到最优解。
- Dantzig于1948年提出了线性规划问题的单纯形算法。
- 单纯形算法的特点是：
- （1）只对约束条件的若干组合进行测试，测试的每一步都使目标函数的值增加；
- （2）一般经过不大于m或n次迭代就可求得最优解。

约束标准型线性规划问题的单纯形算法

- 当线性规划问题中没有不等式约束(8.2)和(8.4)式，而只有等式约束(8.3)和变量非负约束(8.5)时，称该线性规划问题具有标准形式。
- 为便于讨论，不妨先考察一类更特殊的*标准形式线性规划问题*。这一类线性规划问题中，每一个等式约束中，至少有一个变量的系数为正，且这个变量只在该约束中出现。(x1,x4,x6，后页图示)
- 在每一约束方程中选择一个这样的变量，并以它作为变量求解该约束方程。这样选出来的变量称为左端变量或基本变量，其总数为m个。剩下的n-m个变量称为右端变量或非基本变量。
- 这一类特殊的标准形式线性规划问题称为约束标准型线性规划问题。
- 虽然约束标准型线性规划问题非常特殊，但是对于理解线性规划问题的单纯形算法是非常重要的。
- 稍后将看到，任意一个线性规划问题可以转换为约束标准型线性规划问题。

非负约束(8.5)

右端为0，得到一个基本可行解

x_2和x_5为0，与x_3为0的区别

$$\max \quad z = -x_2 + 3x_3 - 2x_5$$

$$\begin{aligned} x_1 + 3x_2 - x_3 + 2x_5 &= 7 \\ x_4 - 2x_2 + 4x_3 &= 12 \\ x_6 - 4x_2 + 3x_3 + 8x_5 &= 10 \end{aligned}$$

$$x_i \geq 0 \qquad i = 1,2,3,4,5,6$$

每一个等式约束中，至少有一个变量的系数为正且这个变量只在该约束中出现

右端变量或非基本变量

左端变量或基本变量

		x_2	x_3	x_5
z	0	-1	3	-2
x_1	7	3	-1	2
x_4	12	-2	4	0
x_6	10	-4	3	8

基本可行解x=(7,0,0,12,0,10)。

- 任何约束标准型线性规划问题，只要将所有非基本变量都置为0，从约束方程式中解出满足约束的基本变量的值，可求得一个***基本可行解***。
- 单纯形算法的基本思想就是**从一个基本可行解出发**，进行一系列的基本可行解的变换。
- 每次变换将一个非基本变量与一个基本变量互调位置，且保持当前的线性规划问题是一个与原问题完全等价的标准线性规划问题。
- 基本可行解x=(7,0,0,12,0,10)。

- **单纯形算法的第1步：**选出使目标函数增加的非基本变量作为**入基变量**。
- 查看单纯形表的第1行（也称之为z行）中标有非基本变量的各列中的值。
- 选出使目标函数增加的非基本变量作为入基变量。
- **z行中的正系数**非基本变量都满足要求。
- 在上面单纯形表的z行中只有1列为正，即非基本变量相应的列，其值为3。
- **选取非基本变量x_3作为入基变量**。

- **单纯形算法的第2步：**选取**离基变量**。
- 在单纯形表中考察由第1步选出的入基变量所相应的列。
- 在一个基本变量变为负值之前，入基变量可以增到多大。
- 因此负元素不考虑（当入基变量变大时，基本变量肯定越变越大，考虑正数）

- 如果入基变量所在的列与基本变量所在行交叉处的表元素为负数，那么该元素将不受任何限制，相应的基本变量只会越变越大。
- 如果入基变量所在列的所有元素都是负值，则目标函数无界，已经得到了问题的无界解。
- 如果选出的列中有一个或多个元素为正数，要弄清是哪个数限制了入基变量值的增加。(在基本变量变负前，p276)
- 受限的增加量可以用入基变量所在列的元素（称为主元素）来除主元素所在行的“常数列”（最左边的列）中元素而得到。所得到数值越小说明受到限制越多。
- 应该选取受到限制最多的基本变量作为离基变量，才能保证将入基变量与离基变量互调位置后，仍满足约束条件。
- 上例中，惟一的一个值为正的z行元素是3，它所在列中有2个正元素，即4和3。
- min{12/4,10/3}=3，应该选取x_4为离基变量；入基变量x_3取值为3。（迭代快慢）

- **单纯形算法的第3步：转轴变换。**
- 转轴变换的目的是将入基变量与离基变量互调位置。
- 给入基变量一个增值，使之成为基本变量；
- 修改离基变量，让入基变量所在列中，离基变量所在行的元素值减为零，而使之成为非基本变量。

$$\max \quad z = -x_2 + 3x_3 - 2x_5$$

$$\begin{aligned} x_1 + 3x_2 - x_3 + 2x_5 &= 7 \\ x_4 - 2x_2 + 4x_3 &= 12 \\ x_6 - 4x_2 + 3x_3 + 8x_5 &= 10 \end{aligned}$$

$$x_i \geq 0 \qquad i = 1,2,3,4,5,6$$

<-原理：系数为负的为0，所有系数为负可以使得z最大

min{12/4,10/3}=3

为了将x_3换成负系数的x_4

见下页

		x_2	x_3	x_5
z	0	-1	3	-2
x_1	7	3	-1	2
x_4	12	-2	4	0
x_6	10	-4	3	8

基本可行解x=(7,0,0,12,0,10)。

$$x_3 - \frac{1}{2}x_2 + \frac{1}{4}x_4 = 3$$

- 解离基变量所相应的方程，将入基变量x_3用离基变量x_4表示为
- 再将其代入其他基本变量和所在的行中消去x_3，

$$\begin{aligned} x_1 + 3x_2 - x_3 + 2x_5 &= 7 \\ x_4 - 2x_2 + 4x_3 &= 12 \\ x_6 - 4x_2 + 3x_3 + 8x_5 &= 10 \end{aligned}$$

$$x_1 + \frac{5}{2}x_2 + \frac{1}{4}x_4 + 2x_5 = 10$$

$$x_6 - \frac{5}{2}x_2 - \frac{3}{4}x_4 + 8x_5 = 1$$

$$z = 9 + \frac{1}{2}x_2 - \frac{3}{4}x_4 - 2x_5$$

- 代入目标函数得到
- 形成新单纯形表

		x_2	x_4	x_5
z	9	1/2	-3/4	-2
x_1	10	5/2	1/2	2
x_3	3	-1/2	1/4	0
x_6	1	-5/2	-3/4	8

- **单纯形算法的第4步：转回并重复第1步，进一步改进目标函数值。**
- **不断重复上述过程，直到z行的所有非基本变量系数都变成负值为止。这表明目标函数不可能再增加了。**
- **在上面的单纯形表中，惟一的值为正的z行元素是非基本变量x_2相应的列，其值为1/2。**
- **因此，选取非基本变量x_2作为入基变量。**
- **它所在列中有惟一的正元素5/2，即基本变量x_1相应行的元素。**
- **因此，选取x_1为离基变量。**
- **再经步骤3的转轴变换得到新单纯形表。**
- **新单纯形表z行的所有非基本变量系数都变成负值，求解过程结束。**
- **整个问题的解可以从最后一张单纯形表的常数列中读出。**
- **目标函数的最大值为11；**
- **最优解为：x*=(0,4,5,0,0,11)。**

$$\max \quad z = -\frac{1}{5}x_1 - \frac{4}{5}x_4 - \frac{12}{5}x_5 + 11$$

	x_1	x_4	x_5
z	11	-1/5	-4/5	-12/5
x_2	4	5/2	1/10	4/5
x_3	5	1/5	3/10	2/5
x_6	11	1	-1/2	10

单纯形算法计算步骤

- 单纯形算法的计算过程可以用单纯形表的形式归纳为一系列基本矩阵运算。
- 主要运算为转轴变换，该变换类似解线性方程组的高斯消去法中的消元变换。
- **单纯形表：**
- $x_1, x_2, \cdots, x_m$ 为基本变量，$x_{m+1}, x_{m+2}, \cdots, x_n$ 为非基本变量。
- 基本变量下标集为B={1,2,...,m}； 非基本变量下标集为N={m+1,m+2,...,n}；
- 当前基本可行解为（$b_1, b_2, \cdots, b_m, 0, \cdots, 0$ ）。

		x_{m+1}	x_{m+2}	…	x_n
z	c_0	c_{m+1}	c_{m+2}	…	c_n
x_1	b_1	a_{1m+1}	a_{1m+2}	…	a_{1n}
x_2	b_2	a_{2m+1}	a_{2m+2}	…	a_{2n}
⋮	⋮				
x_m	b_m	a_{mm+1}	a_{mm+2}	…	a_{mn}

- **单纯形算法计算步骤**如下：
- **步骤1：选入基变量。（找目标函数中正系数）**
- 如果所有$c_j \le 0$，则当前基本可行解为最优解，计算结束。
- 否则取$c_e > 0$相应的非基本变量x_e为入基变量。
- **步骤2：选离基变量。（找约束方程中可替换的负系数）**
- 对于步骤1选出的入基变量x_e，如果所有$a_{ie} \le 0$，则最优解无界，计算结束。
- 否则计算

$$\theta = \min_{a_{ie}>0}\left\{\frac{b_i}{a_{ie}}\right\} = \frac{b_k}{a_{ke}}$$

- 选取基本变量x_k为离基变量。
- 新的基本变量下标集为 $\overline{B} = B + \{e\} - \{k\}$
- 新的非基本变量下标集为 $\overline{N} = N + \{k\} - \{e\}$
- **步骤3：作转轴变换。**
- 新单纯形表中各元素变换如下。

- $$\begin{cases} \overline{b}_i = b_i - a_{ie}\dfrac{b_k}{a_{ke}} & i \in \overline{B} \\ \overline{b}_e = \dfrac{b_k}{a_{ke}} \end{cases} \qquad (8.10)$$

- $$\begin{cases} \overline{a}_{ij} = a_{ij} - a_{ie}\dfrac{a_{kj}}{a_{ke}} & i \in \overline{B},\quad j \in \overline{N} \\ \overline{a}_{ik} = -\dfrac{a_{ie}}{a_{ke}} \end{cases} \qquad (8.11)$$

- $$\begin{cases} \overline{a}_{ej} = \dfrac{a_{kj}}{a_{ke}} & j \in \overline{N} \\ \overline{a}_{ek} = \dfrac{1}{a_{ke}} \end{cases} \qquad (8.12)$$

- $$\begin{cases} \overline{c}_i = c_i - c_e\dfrac{a_{ki}}{a_{ke}} & i \in \overline{N} \\ \overline{c}_k = -\dfrac{c_e}{a_{ke}} \end{cases} \qquad (8.13)$$

- **步骤4：**转步骤1。

将一般问题转化为约束标准型

- 有几种巧妙的办法可以将一般的线性规划问题转换为约束标准型线性规划问题。
- 首先，需要把(8.2)或(8.4)形式的不等式约束转换为等式约束。
- 具体做法是，引入**松弛/剩余变量**，利用松弛变量的非负性，将不等式转化为等式。
- **松驰变量记为y_i，共有m_1+m_3个**。
- 在求解过程中，应当将松弛变量与原来变量同样对待。求解结束后，抛弃松弛变量。
- 注意松弛变量前的符号由相应的原不等式的方向所确定。

$$\begin{aligned} x_1 + 2x_3 &\le 18 \\ 2x_2 - 7x_4 &\le 0 \\ x_1 + x_2 + x_3 + x_4 &= 9 \\ x_2 - x_3 + 2x_4 &\ge 1 \end{aligned}$$

$$\begin{aligned} x_1 + 2x_3 + y_1 &= 18 \\ 2x_2 - 7x_4 + y_2 &= 0 \\ x_1 + x_2 + x_3 + x_4 &= 9 \\ x_2 - x_3 + 2x_4 - y_3 &= 1 \end{aligned}$$

- 为了进一步构造标准型约束，还需要引入m个人工变量，记为z_i。
- 至此，原问题已经变换为等价的约束标准型线性规划问题。
- 对极小化线性规划问题，只要将目标函数乘以-1即可化为等价的极大化线性规划问题。

$$
\begin{aligned}
z_1 + x_1 + 2x_3 + y_1 &= 18 \\
z_2 + 2x_2 - 7x_4 + y_2 &= 0 \\
z_3 + x_1 + x_2 + x_3 + x_4 &= 9 \\
z_4 + x_2 - x_3 + 2x_4 - y_3 &= 1
\end{aligned}
$$

略

一般线性规划问题的2阶段单纯形算法

- 引入人工变量后的线性规划问题与原问题并不等价，除非所有z_i都是0 。
- 为了解决这个问题，在求解时必须分2个阶段进行。
- 第一阶段用一个辅助目标函数 $z' = -\sum_{i=1}^{m} z_i$ 替代原来的目标函数。
- 这个线性规划问题称为原线性规划问题所相应的辅助线性规划问题。
- 对辅助线性规划问题用单纯形算法求解。
- 如果原线性规划问题有可行解，则辅助线性规划问题就有最优解，且其最优值为0，即所有z_i都为0。（约束z,x,y>=0)

略

一般线性规划问题的2阶段单纯形算法

- 在辅助线性规划问题最后的单纯形表中，所有z_i均为非基本变量。
- 划掉所有z_i相应的列，剩下的就是只含x_i和y_i的约束标准型线性规划问题了。
- 单纯形算法第一阶段的任务就是构造一个初始基本可行解。
- 单纯形算法第二阶段的目标是求解由第一阶段导出的问题。
- 此时要用原来的目标函数进行求解。
- 如果在辅助线性规划问题最后的单纯形表中， z_i不全为0，则原线性规划问题没有可行解，从而原线性规划问题无解。

略

退化情形的处理

- 用单纯形算法解一般的线性规划问题时，可能会遇到退化的情形，即在迭代计算的某一步中，常数列中的某个元素的值变成0，使得相应的基本变量取值为0。
- 如果选取退化的基本变量为离基变量，则作转轴变换前后的目标函数值不变。在这种情况下，算法不能保证目标函数值严格递增，因此，可能出现无限循环。
- 考察下面的由Beale在1955年提出的退化问题的例子。
- 按照2阶段单纯形算法求解该问题将出现无限循环。

$$\max \quad z = \frac{3}{4}x_1 - 20x_2 + \frac{1}{2}x_3 - 6x_4$$

$$\begin{aligned} \frac{1}{4}x_1 - 8x_2 - x_3 + 9x_4 &\le 0 \\ \frac{1}{2}x_1 - 12x_2 - \frac{1}{2}x_3 + 3x_4 &\le 0 \\ x_3 &\le 1 \end{aligned}$$

$$x_i \ge 0 \qquad i = 1,2,3,4$$

略

退化情形的处理

- 在单纯形法中，基变量一般都取非零值，非基变量都取零值。如果某个基可行解中存在取零值的基变量，则称该解为退化解。在退化情况下，如果取退化的基变量为离基变量，则变化后的解仍为退化解，且目标函数值不变。在以后的迭代中，如果每次都取退化的基变量为离基变量，则迭代可能只在可行域的几个顶点中间反复进行，即出现计算过程的循环，而达不到最优解。但实际中，循环现象极为罕见，因此，我们计算的时候完全可以不考虑循环问题。

略

- Bland提出避免循环的一个简单易行的方法。
- Bland提出在单纯形算法迭代中，按照下面的2个简单规则就可以避免循环。

- **规则1**：设 $e = \min\{j \mid c_j > 0\}$，取$x_e$为入基变量。

- **规则2**：设 $k = \min\left\{l \mid \frac{b_l}{a_{le}} = \min_{a_{ie}>0}\left\{\frac{b_i}{a_{ie}}\right\}\right\}$

- 取x_k为离基变量。

- 算法leave(col)已经按照规则2选取离基变量。
- 选取入基变量的算法enter(objrow) 中只要加一个break语句即可。

略

仓库租赁问题

- 某企业计划为流通的货物租赁一批仓库。必须保证在时间段i=1,2,…,n，有b_i的仓库容量可用。现有若干仓库源可供选择。设c_{ij}是从时间段i到时间段j租用1个单位仓库容量的价格，$1 \le i \le j \le n$。应如何安排仓库租赁计划才能满足各时间段的仓库需求，且使租赁费用最少。
- 设租用时间段i到时间段j的仓库容量为y_{ij}，$1 \le i \le j \le n$。则租用仓库的总费用为：$\sum_{i=1}^{n}\sum_{j=i}^{n} c_{ij} y_{ij}$
- 在时间段k可用的仓库容量为：$\sum_{i=1}^{k}\sum_{j=k}^{n} y_{ij}$
- 仓库租赁问题可表述为下面的线性规划问题：

$$\min \quad \sum_{i=1}^{n}\sum_{j=i}^{n} c_{ij} y_{ij}$$

$$\sum_{i=1}^{k}\sum_{j=k}^{n} y_{ij} \ge b_k$$

$$y_{ij} \ge 0$$

略

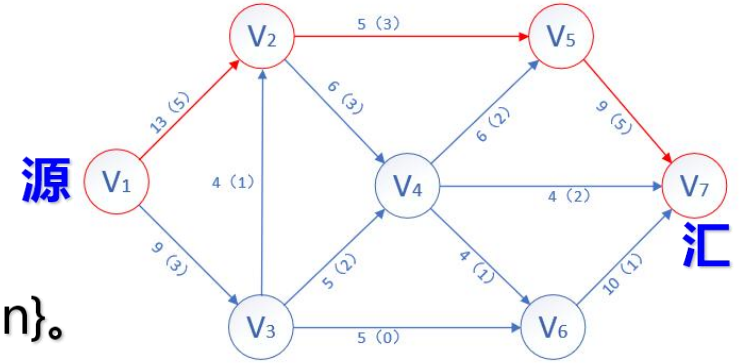
- 设 m=n(n+1)/2;
- $(\ y_{11},y_{12},\cdots,y_{1n},y_{22},y_{23},\cdots,y_{2n},\cdots,y_{nn}\ ) = (\ x_1,x_2,\cdots,x_m\ )$;
- $(\ c_{11},c_{12},\cdots,c_{1n},c_{22},c_{23},\cdots,c_{2n},\cdots,c_{nn}\ ) = (\ d_1,d_2,\cdots,d_m\ )$;
- 上述线性规划问题可表述为n个约束和m个变量的标准线性规划问题如下。

$$\min \quad d^T x$$
$$Ax \geq b$$
$$x \geq 0$$

$$A=\begin{bmatrix} 1 & 1 & \cdots & 1 & 1 & 0 & 0 & \cdots & 0 & 0 & 0 & \cdots & 0 \\ 0 & 1 & \cdots & 1 & 1 & 1 & 1 & \cdots & 1 & 0 & & & \\ 0 & 0 & 1 & \cdots & 1 & 0 & 1 & \cdots & 1 & \vdots & \vdots & \vdots & \vdots \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots & 0 & 0 & 0 & 0 \\ 0 & 0 & \cdots & 0 & 1 & 0 & \cdots & 0 & 1 & 0 & \cdots & 0 & 1 \end{bmatrix}$$

略

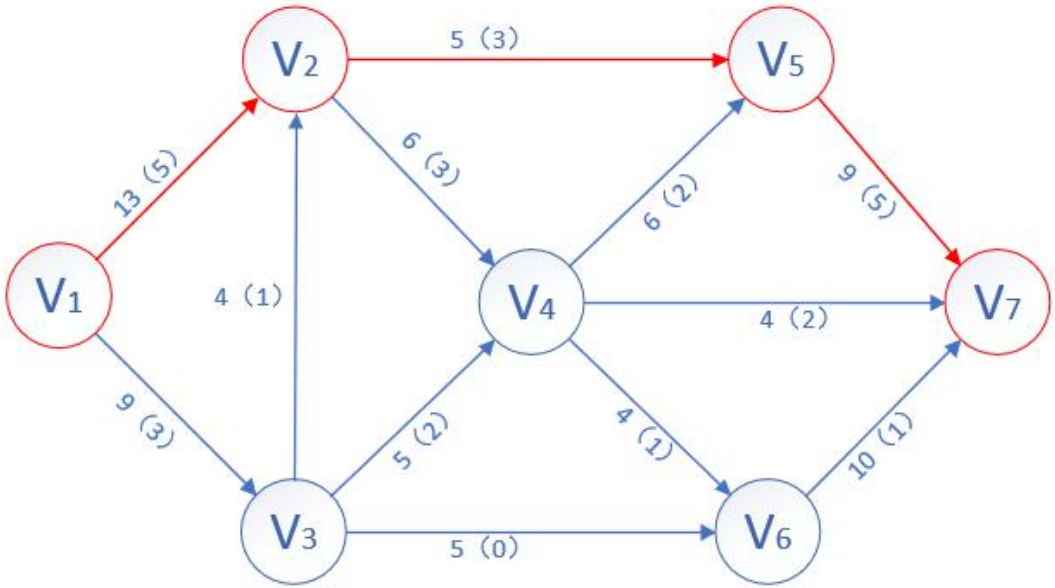
8.2 最大网络流问题



- **1 基本概念和术语**
- **（1） 网络（有源有汇有容量的图）**
- G是一个简单有向图，G=(V,E)，V={1，2，…，n}。
- 在V中指定一个顶点s，称为**源**和另一个顶点t，称为**汇**。
- 有向图G的每一条边(v,w)∈E，对应有一个值cap(v,w)≥0，称为边的**容量**。
- 这样的有向图G称作一个**网络**。
- **（2） 网络流**
- 网络上的**流**是定义在网络的边集合E上的一个非负函数flow={flow(v,w)}，并称flow(v,w)为边(v,w)上的**流量**。

8.2 最大网络流问题

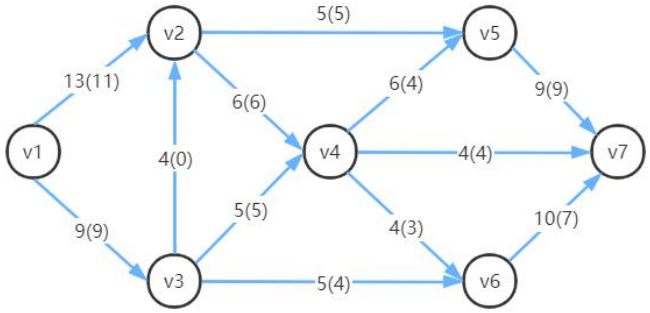
- 例1 如图连结某产品产地V_s和销地V_t的交通图。弧(V_i,V_j)表示从V_i到V_j的运输线，弧旁的数字表示这条运输线的最大通过能力C_{ij}，括号内的数字表示该弧上的实际流F_{ij}。现要求制定一个运输方案，使从V_s运到V_t的产品数量最多。

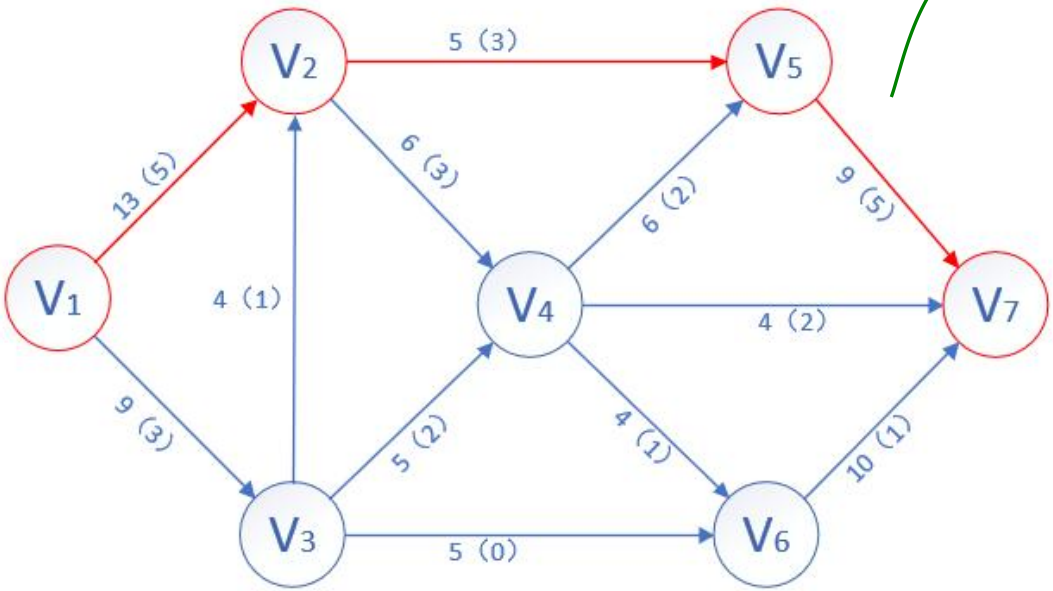


一是每条弧上的流量必须是非负的且不能超过该弧的最大通过能力（即该弧的容量）；
二是起点发出的流的总和（称为流量），必须等于终点接收的流的总和，
且各中间点流入的流量之和必须等于从该点流出的流量之和，即流入的流量之和与流出的流量之和的差为零，也就是说各中间点只起转运作用，
它既不产出新的物资，也不得截留过境的物资。

8.2 最大网络流问题

- 上例网络最大流问题的线性规划模型
- 设f={f_{ij}}，则其线性规划模型为maxV(f)





$$
s.t.\begin{cases}
f_{s2}+f_{s3}=v(f)\\
f_{s2}+f_{32}-f_{25}-f_{24}=0\\
f_{s3}-f_{32}-f_{34}-f_{36}=0\\
f_{24}+f_{34}-f_{45}-f_{46}-f_{4t}=0\\
f_{25}+f_{45}-f_{5t}=0\\
f_{36}+f_{46}-f_{6t}=0\\
f_{5t}+f_{4t}+f_{6t}=v(f)\\
0\le f_{ij}\le c_{ij}
\end{cases}
$$

(3) 可行流

满足下述条件的流flow称为**可行流**：

- **(3.1)容量约束：**对每一条边(v,w)∈E，0≤flow(v,w)≤cap(v,w)。
- 注意：0流是可行流

- **(3.2)平衡约束：**
- **对于中间顶点：流出量=流入量。**
- **即对每个v∈V(v≠s,t)有：顶点v的流出量 - 顶点v的流入量=0，即**

$$\sum_{(v,w)\in E} flow(v,w) - \sum_{(w,v)\in E} flow(w,v) = 0$$

- **对于源s：s的流出量 - s的流入量=源的净输出量f，即**

$$\sum_{(s,v)\in E} flow(s,v) - \sum_{(v,s)\in E} flow(v,s) = f$$

- **对于汇t：t的流入量 - t的流出量的=汇的净输入量f，即**

$$\sum_{(v,t)\in E} flow(v,t) - \sum_{(t,v)\in E} flow(t,v) = f$$

- **式中f 称为这个可行流的流量，即源的净输出量(或汇的净输入量)。**
- **可行流总是存在的。例如，让所有边的流量flow(v,w)=0，就得到一个其流量f=0的可行流(称为0流)。**

- （4） 边流
- 对于网络G的一个给定的可行流flow，
- ✓ 将网络中满足flow(v,w)=cap(v,w)的边称为饱和边；
- ✓ flow(v,w)<cap(v,w)的边称为非饱和边；
- ✓ flow(v,w)=0的边称为零流边；
- ✓ flow(v,w)>0的边称为非零流边。
- ✓ 当边(v,w)既不是一条零流边也不是一条饱和边时，称为弱流边。
- （5） 最大流
- 最大流问题即求网络G的一个可行流flow，使其流量f达到最大。即flow满足：
- $0 \le flow(v,w) \le cap(v,w)$，$(v,w) \in E$；且

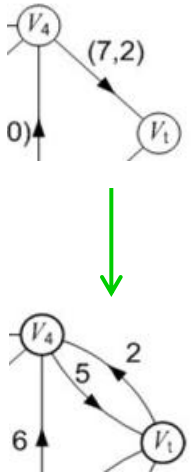
$$\sum flow(v,w) - \sum flow(w,v) = \begin{cases} f & v = s \\ 0 & v \ne s,t \\ -f & v = t \end{cases}$$

- **（6） 流的费用**
- **在实际应用中，与网络流有关的问题，不仅涉及流量，而且还有费用的因素。此时网络的每一条边(v,w)除了给定容量cap(v,w)外，还定义了一个单位流量费用cost(v,w)。对于网络中一个给定的流flow，其费用定义为：**

$$\cos t(flow) = \sum_{(v,w)\in E} \cos t(v,w) \times flow(v,w)$$

- （7） 残流网络
- 对于给定的一个流网络G及其上的一个流flow，网络G关于流flow的残流网络G*与G有相同的顶点集V，而网络G中的每一条边对应于G*中的1条边或2条边。

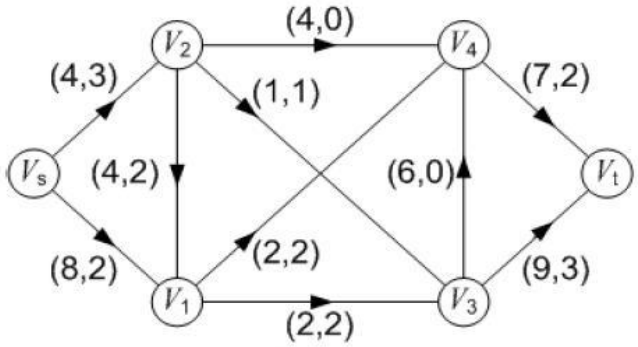
- 设(v,w)是G的一条边。
- 当flow(v,w)>0时，（w,v）是G*中的一条边，该边的容量为cap*(w,v)=flow(v,w)；
- 当flow(v,w)<cap(v,w)时，(v,w)是G*中的另一条边，该边的容量为
- cap*(v,w)=cap(v,w)-flow(v,w)。
- 【未表现0边】



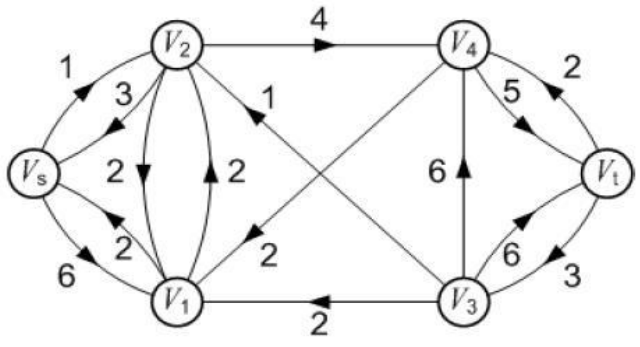
- 残流网络是设计与网络流有关算法的重要工具。(举例)

- **（7） 残流网络**

- **按照残流网络的定义，当原网络G中的边(v,w)是一条零流边时，残流网络G*中有唯一的一条边(v,w)与之对应，且该边的容量为cap(v,w)。**
- **当原网络G中的边(v,w)是一条饱和边时，残流网络G*中有唯一的一条边(w,v)与之对应，且该边的容量为cap(v,w)。 （与1的方向不同）**
- **当原网络G中的边(v,w)是一条弱流边时，残流网络G*中有2条边(v,w)和(w,v)与之对应，这2条边的容量分别为cap(v,w) -flow(v,w)和flow(v,w)。**



(a) 容量网络*G*



(b) 残留网络*G'*

https://blog.csdn.net/jizhihong

增广路算法

- **1 算法基本思想**

- **设P是网络G中联结源s和汇t的一条路。定义路的方向是从s到t。**

- **将路P上的边分成2类：**
- **一类边的方向与路的方向一致，称为向前边。向前边的全体记为P+。**
- **另一类边的方向与路的方向相反，称为向后边。向后边的全体记为P-。**

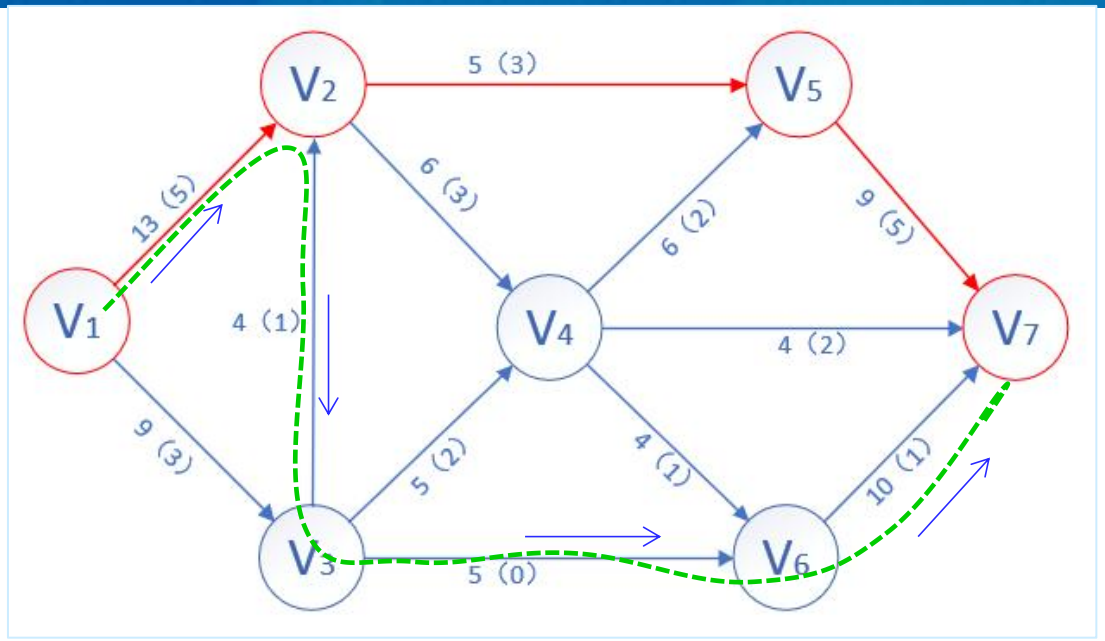
增广路算法

- **1 算法基本思想**

- 设flow是一个可行流，P是从s到t的一条路，若P满足下列条件：
- （1）在P的所有向前边(v,w)上，flow(v,w)<cap(v,w)，即P+中的每一条边都是非饱和边；
- （2）在P的所有向后边(v,w)上，flow(v,w)>0，即P-中的每一条边都是非零流边。
- 则称P为关于可行流flow的一条可增广路。
- 可增广路是残流网络中一条容量大于0的路。

- 将具有上述特征的路P称为可增广路是因为可以通过修正路P上所有边流量flow(v,w)将当前可行流改进成一个流值更大的可行流。

增广路算法



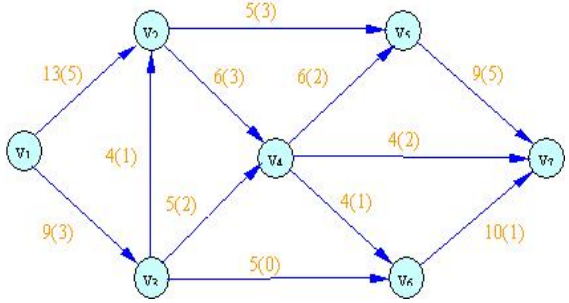
- $P=\{V_s,(V_s,V_2),V_2,(V_3,V_2),V_3(V_3,V_6),V_6,(V_6,V_t),V_t\}$

- $P+=\{(V_s,V_2),(V_3,V_6),(V_6,V_t)\}$
- $P-=\{(V_3,V_2)\}$ //残流网络

- 增流的具体做法是：
- （1）不属于可增广路P的边(v,w)上的流量保持不变；
- （2）可增广路P上的所有边(v,w)上的流量按下述规则变化：
- 在向前边(v,w)上，flow(v,w)+d；
- 在向后边(v,w)上，flow(v,w)-d。
- 按下面的公式修改当前的流。

$$flow(v,w)=\begin{cases} flow(v,w)+d & (v,w)\in P^{+} \\ flow(v,w)-d & (v,w)\in P^{-} \\ flow(v,w) & (v,w)\notin P \end{cases}$$

- 其中d称为可增广量，可按下述原则确定：d取得尽量大，又要使变化后的流仍为可行流。
- 按照这个原则，d既不能超过每条向前边(v,w)的cap(v,w)-flow(v,w)，也不能超过每条向后边(v,w)的flow(v,w)。
- 因此d应该等于向前边上的cap(v,w)-flow(v,w)与向后边上的flow(v,w)的最小值。也就是残流网络中P的最大容量。
- 增广路定理：设flow是网络G的一个可行流，如果不存在从s到t关于flow的可增广路P，则flow是G的一个最大流。
- 实例讲解(Flash演示)



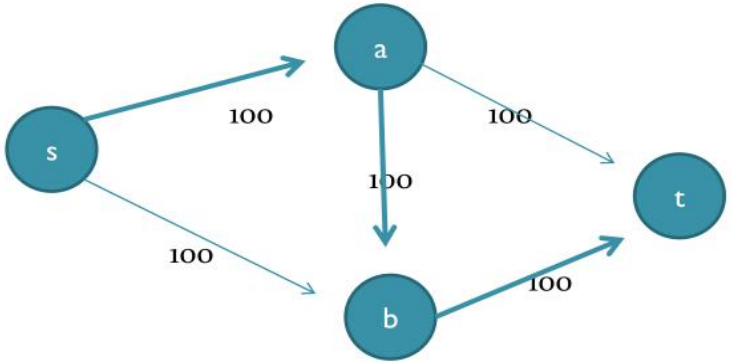
- **2 算法描述**
- 最大流的增广路算法如下，该算法也常称作Ford Fulkerson算法。（标号法）

```
template <class Graph, class Edge> class MAXFLOW
{   const Graph &G;
    int s, t,maxf;
    vector<int> wt;
    vector<Edge *> st;
    int ST(int v) const { return st[v]->other(v); }
    void augment(int s, int t)
    {   int d = st[t]->capRto(t);
        for (int v = ST(t); v != s; v = ST(v))
          if (st[v]->capRto(v) < d)  d = st[v]->capRto(v);
        st[t]->addflowRto(t, d);
        maxf+=d;
        for ( v = ST(t); v != s; v = ST(v))  st[v]->addflowRto(v, d);
    }
    bool pfs();
public:
     MAXFLOW(const Graph &G, int s, int t,int &maxflow) :
         G(G), s(s), t(t), st(G.V()), wt(G.V()),maxf(0)
     {   while (pfs()) augment(s, t); maxflow+=maxf;}
};
```

- **3 算法的计算复杂性**
- 增广路算法的效率由下面2个因素所确定。
- （1）整个算法找增广路的次数；
- （2）每次找增广路所需的时间。
- 给定的网络中有n个顶点和m条边，且每条边的容量不超过M。
- 可以证明，在一般情况下，增广路算法中找增广路的次数不超过nM次。
- 最短增广路算法在最坏情况下找增广路的次数不超过nm/2次。
- 找1次增广路最多需要$O(m)$计算时间。
- 因此，在最坏情况下最短增广路算法所需的计算时间为$O(nm^2)$ 。
- 当给定的网络是稀疏网络，即$m=O(n)$时，最短增广路算法所需的计算时间为$O(n^3)$。
- 最大容量增广路算法在最坏情况下找增广路的次数不超过2mlogM次。
- 由于使用堆来存储优先队列，找1次增广路最多需要O(nlogn)计算时间。
- 因此，在最坏情况下最大容量增广路算法所需的计算时间为 $O(mn\log n\log M)$
- 当给定的网络是稀疏网络时，最大容量增广路算法所需的计算时间为 $O(n^2\log n\log M)$

朴素dfs:

使用dfs每次寻找s->t的一条通路，然后将这条路上的流量值定义为其中最小的容量值.。完成这次运输后，将通路上的所有边的容量值减去流量值，开始下一次的寻找，直到没有通路，完成算法。



如果第一次dfs到的通路是:

s - > a - > b - > t,

那么这次之后，网络中就再也没有从s - > t的通路了，

按此计算，这个网络的最大流就是100。

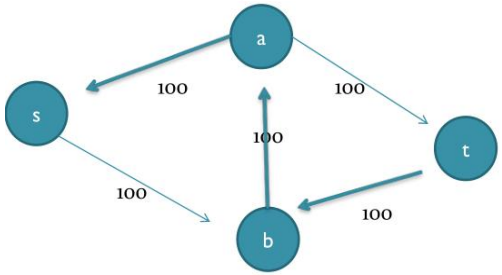
然而，很明显的，这个网络的最大流是200。

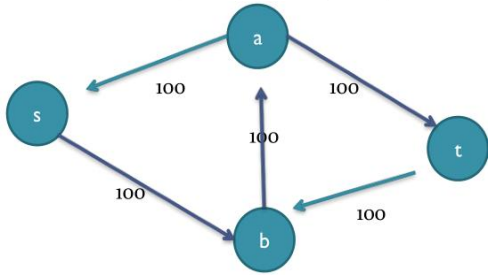
The above implementation of Ford Fulkerson Algorithm is called Edmonds-Karp Algorithm. The idea of Edmonds-Karp is to use BFS in Ford Fulkerson implementation as BFS always picks a path with minimum number of edges. When BFS is used, the worst case time complexity can be reduced to O(VE2). The above implementation uses adjacency matrix representation though where BFS takes O(V2) time, the time complexity of the above implementation is O(EV3) (Refer CLRS book for proof of time complexity)

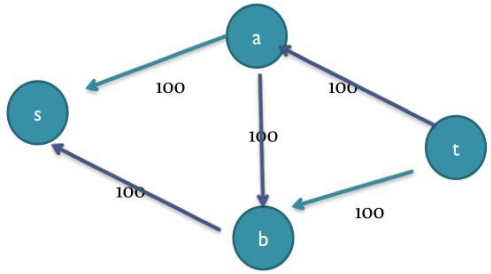
利用残流网络：Edmonds-Karp算法 / BFS的Ford-Fulkerson

对上次dfs 时找到的流量路径上的边，添加一条“反向”边，反向边上的容量等于上次dfs 时找到的该边上的流量，然后再利用“反向”的容量和其他边上剩余的容量寻找路径。

1、第一次dfs依然是s - > a - > b - > t，与上次不同的是，这次添加上反向边。

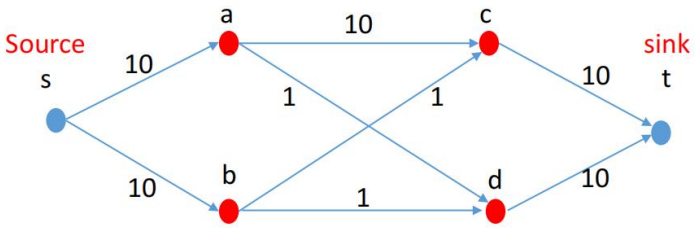




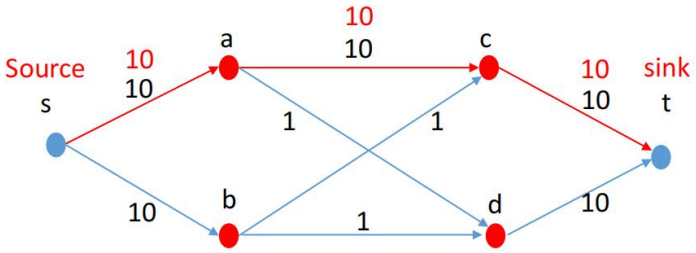


2、继续在新图上找，可以找到另一条流量为100的通路s - > b - > a - > t。

3、对第二次找到的路径添加反向边，图中已经没有从s - > t的通路了，于是该网络的最大流就是200。

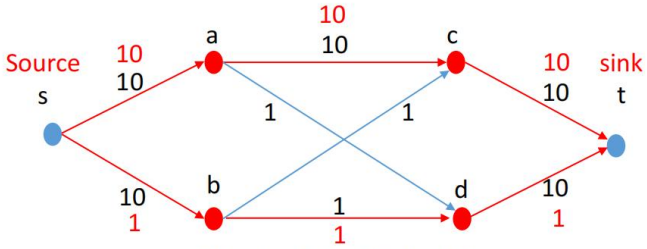


- Flow 1:



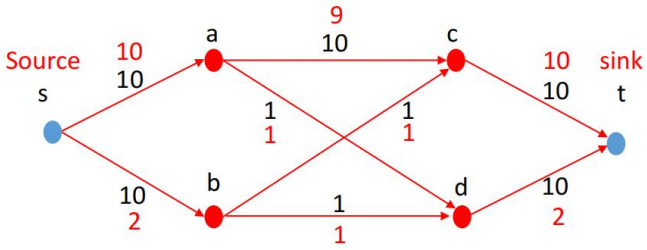
Flow is of size 10

- Flow 2:

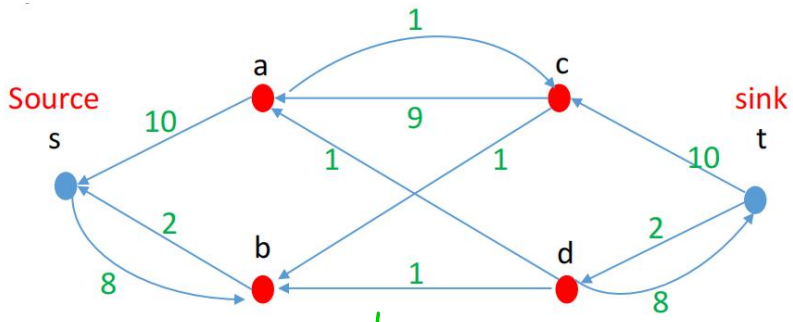


Flow is of size 10+1 = 11

- Flow 3:



Flow is of size 10+2 = 12



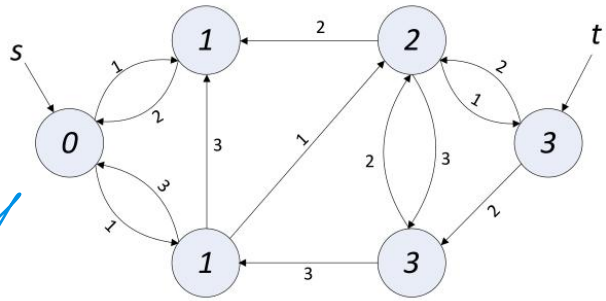
最后残留网络

Not obvious

存在一个反向残留

- Dinic：基于“层次图”的时间效率优先的最大流算法。BFS+DFS
- 层次，就是从源点走到那个点的最短路径长度。
- 从源点开始，在层次图中沿着边不管怎么走，经过的路径一定是终点在剩余图中的最短路。

- 根据残量网络计算层次图。
- 在层次图中使用DFS进行增广直到不存在增广路
- 重复以上步骤直到无法增广



In Edmond's Karp algorithm, we use BFS to find an augmenting path and send flow across this path. In Dinic's algorithm, we use BFS to check if more flow is possible and to construct level graph. In level graph, we assign levels to all nodes, level of a node is shortest distance (in terms of number of edges) of the node from source. Once level graph is constructed, we send multiple flows using this level graph. This is the reason it works better than Edmond Karp. In Edmond Karp, we send only flow that is send across the path found by BFS.

1、使用BFS对残余网络进行分层，在分层时，只要进行到汇点的层次数被算出即可停止，因为按照该DFS的规则，和汇点同层或更下一层的节点，是不可能走到汇点的。

2、分完层后，从源点开始，用DFS从前一层向后一层反复寻找增广路(即要求DFS的每一步都必须要走到下一层的节点）。

3、DFS过程中，要是碰到了汇点，则说明找到了一条增广路径。此时要增加总流量的值，消减路径上各边的容量，并添加反向边，即所谓的进行增广。

4、DFS找到一条增广路径后，并不立即结束，而是回溯后继续DFS寻找下一个增广路径。回溯到的结点满足以下的条件：

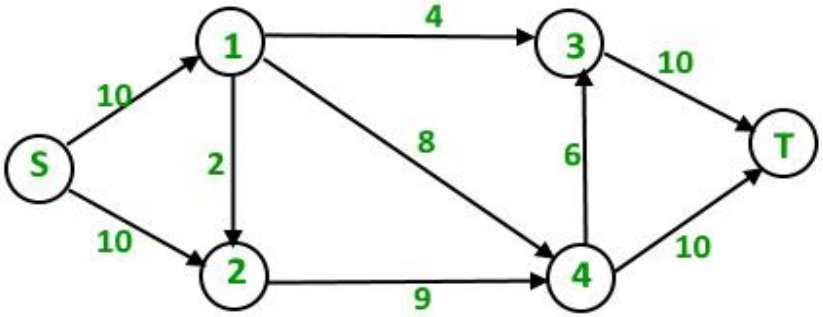
1) DFS搜索树的树边(u,v)上的容量已经变成0。即刚刚找到的增广路径上所增加的流量，等于(u,v)本次增广前的容量。(DFS的过程中，是从u走到更下层的v的)

2) u是满足条件 1)的最上层的节点。

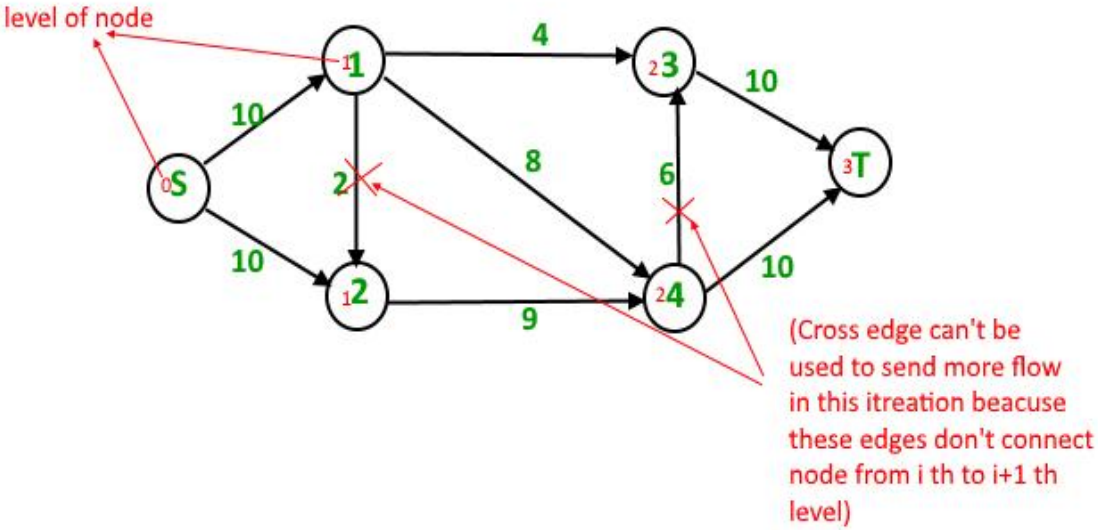
5、如果回溯到源点而且无法继续往下走了，DFS结束。因此，一次DFS过程中，可以找到多条增广路径。

6、DFS结束后，对残余网络再次进行分层，然后再进行DFS当残余网络的分层操作无法算出汇点的层次（即BFS到达不了汇点）时，算法结束，最大流求出

In Edmond's Karp algorithm, we use BFS to find an augmenting path and send flow across this path. In Dinic's algorithm, we use BFS to check if more flow is possible and to construct level graph. In level graph, we assign levels to all nodes, level of a node is shortest distance (in terms of number of edges) of the node from source. Once level graph is constructed, we send multiple flows using this level graph. This is the reason it works better than Edmond Karp. In Edmond Karp, we send only flow that is send across the path found by BFS.

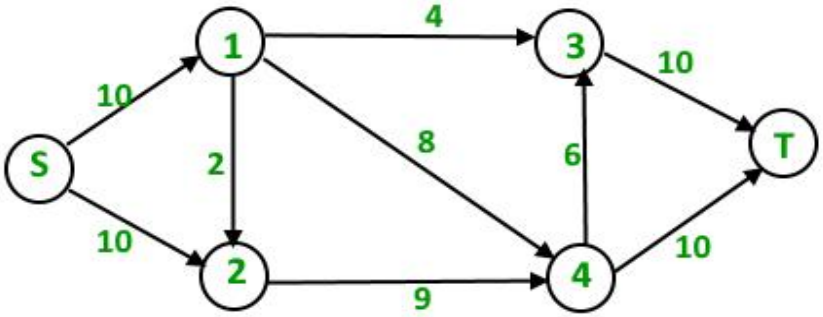


原网络



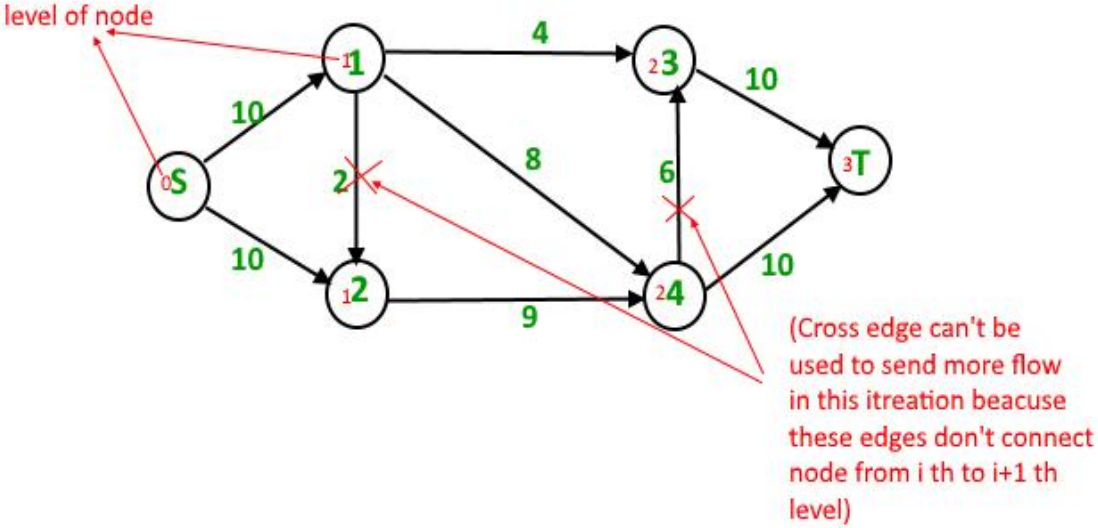
层次图

In Edmond's Karp algorithm, we use BFS to find an augmenting path and send flow across this path. In Dinic's algorithm, we use BFS to check if more flow is possible and to construct level graph. In level graph, we assign levels to all nodes, level of a node is shortest distance (in terms of number of edges) of the node from source. Once level graph is constructed, we send multiple flows using this level graph. This is the reason it works better than Edmond Karp. In Edmond Karp, we send only flow that is send across the path found by BFS.

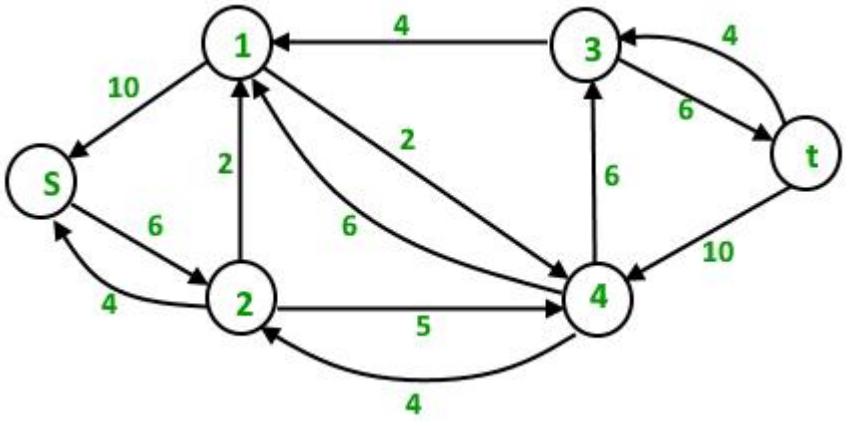


第1次迭代：

4 units of flow on path s – 1 – 3 – t.
6 units of flow on path s – 1 – 4 – t.
4 units of flow on path s – 2 – 4 – t.
Total flow = Total flow + 4 + 6 + 4 = 14



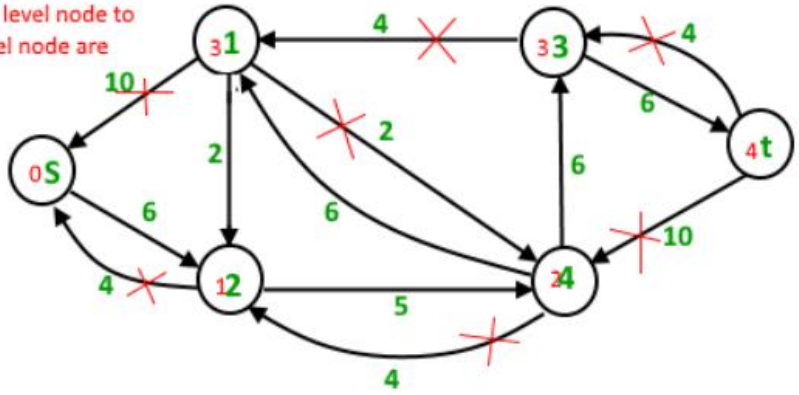
In Edmond's Karp algorithm, we use BFS to find an augmenting path and send flow across this path. In Dinic's algorithm, we use BFS to check if more flow is possible and to construct level graph. In level graph, we assign levels to all nodes, level of a node is shortest distance (in terms of number of edges) of the node from source. Once level graph is constructed, we send multiple flows using this level graph. This is the reason it works better than Edmond Karp. In Edmond Karp, we send only flow that is send across the path found by BFS.

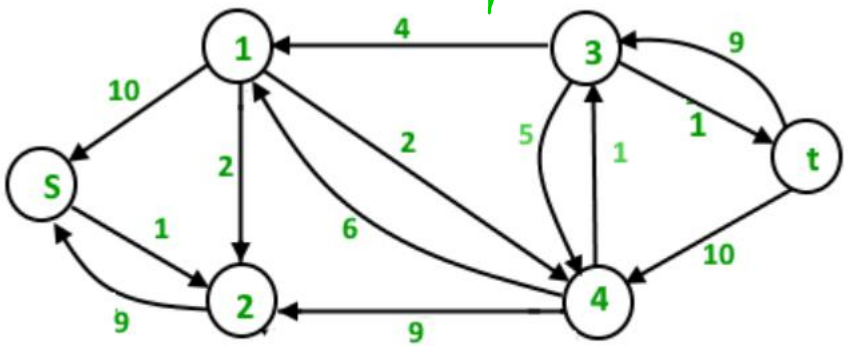


第2次迭代:

5 units of flow on path s – 2 – 4 – 3 – t
Total flow = Total flow + 5 = 19

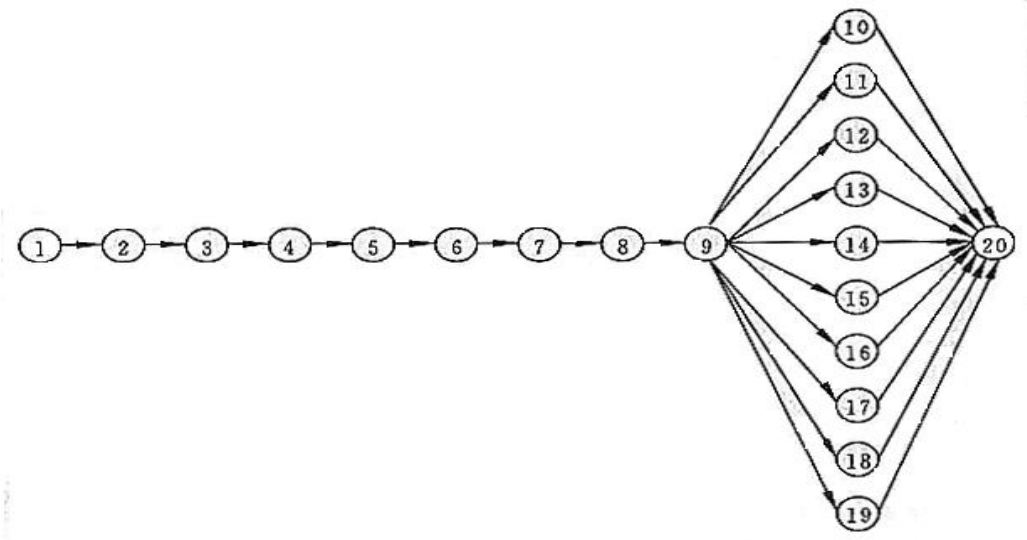
Those edges that are not go from i th level node to (i+1)th nlevel node are crossed

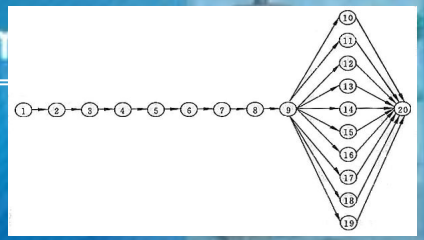




预流推进算法

- **1 算法基本思想**
- 增广路算法的特点是找到增广路后，立即沿增广路对网络流进行增广。
- 每一次增广可能需要对最多n-1条边进行操作。
- 最坏情况下，每一次增广需要$O(n)$计算时间。
- 有些情况下，这个代价是很高的。下面是一个极端的例子。
- s=1,t=20; 1-9的边容量为10，其他边容量为1。





- 无论用哪种增广路算法，都会找到10条增广路，每条路长为10，容量为1。
- 共需要10次增广，每次增广需要对10条边进行操作，每条边增广1个单位流量。
- 10条增广路中的前9个顶点（前8条边）是完全一样的。
- 如果直接将前8条边的流量增广10个单位，而只对后面长为2的不同的有向路单独操作，就可以节省许多计算时间。
- 这就是预流推进（preflow push)算法的基本思想。
- 预流推进算法注重对每一条边的增流，而不必每次一定对一条增广路增流。
- 通常将沿一条边增流的运算称为一次推进（push）。
- 在算法的推进过程中，网络流满足容量约束，但一般不满足流量平衡约束。
- 从每个顶点（除s和t外）流出的流量之和总是小于等于流入该顶点的流量之和。
- 这种流称为预流（preflow）。这也是这类算法被称为预流推进算法的原因。
- 下面先给出预流的严格定义。
- 给定网络G=(V,E)一个预流是定义在G的边集E上的一个正边流函数。
- 该函数满足容量约束，即对G的每一条边(v,w)∈E，满足$0 \le flow(v,w) \le cap(v,w)$。

- G的每一中间顶点满足流出量小于或等于流入量。$\sum_{(v,w)\in E} flow(v,w) \le \sum_{(w,v)\in E} flow(w,v)$
- 即对每个v∈V(v≠s,t)有
- 满足条件 $\sum_{(v,w)\in E} flow(v,w) < \sum_{(w,v)\in E} flow(w,v)$ 的中间顶点v称为活顶点。
- 量 $\sum_{(w,v)\in E} flow(w,v) - \sum_{(v,w)\in E} flow(v,w)$ 称为顶点v的存流。
- 按此定义，源s和汇t不可能成为活顶点。
- 对网络G上的一个预流，如果存在活顶点，则说明该预流不是可行流。
- 预流推进算法就是要选择活顶点，并通过把一定的流量推进到它的邻点，尽可能地将当前活顶点处正的存流减少为0，直至网络中不再有活顶点，从而使预流成为可行流。
- 如果当前活顶点有多个邻点，那么首先推进到哪个邻点呢?
- 由于算法最后的目的是尽可能将流推进到汇点t，因此算法应寻求把流量推进到它的邻点中距顶点t最近的顶点。
- 预流推进算法中用到一个高度函数h来确定推流边。
- 对于给定网络G=(V,E)的一个流，其高度函数h是定义在G的顶点集V上的一个非负函数。该函数满足：
- （1）对于G的残流网络中的每一条边(u,v)有，h(u) ≤ h(v)+1；（2）h(t)=0。
- G的残流网络中满足h(u) = h(v)+1的边(u,v)称为G的可推流边。

可行流要达到平衡

一般的预流推进算法

步骤0： 构造初始预流flow：

对源顶点s的每条出边(s,v)令flow(s,v)=cap(s,v)；

对其余边(u,v)令flow(u,v)=0。构造一有效的高度函数h。

步骤1： 如果残量网络中不存在活顶点，则计算结束，已经得到最大流；

否则转步骤2。

步骤2： 在网络中选取活顶点v。

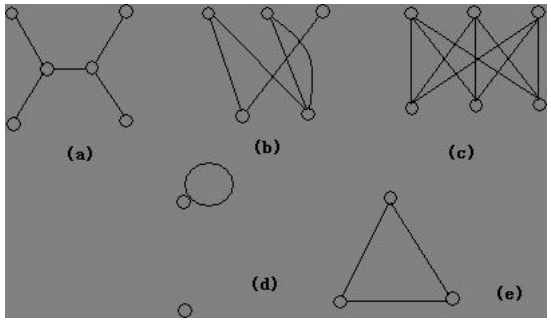
如果存在顶点v的出边为可推流边，则选取一条这样的可推流边，并沿此边推流。

否则令h(v) = min{h(w)+1 | (v,w)是当前残流网络中的边}，并转步骤1。

- 一般的预流推进算法的每次迭代是一次推进运算或者一次高度重新标号运算。
- 如果推进的流量等于推流边上的残留容量，则称为**饱和推进**，否则称为**非饱和推进**。
- 算法终止时，网络中不含有活顶点。此时只有顶点s和t的存流非零。此时的预流实际上已经是一个可行流。
- 算法预处理阶段已经令h(s)=n，而高度函数在计算过程中不会减少，因此算法在计算过程中可以保证网络中不存在增广路。
- 根据增广路定理，**算法终止时的可行流是一个最大流**。

- 一般的预流推进算法并未给出如何选择活顶点和可推流边。
- 不同的选择策略导致不同的预流推进算法。
- 在基于顶点的预流推进算法中，选定一个活顶点后，算法沿该活顶点的所有推流边进行推流运算，直至无可推流边或该顶点的存变成0时为止。
- **3 算法的计算复杂性**
- 基于顶点的预流推进算法用一个**广义队列gQ存储当前活顶点集合**。
- 广义队列可以是通常的FIFO队列，LIFO栈，随机化队列，随机化栈，或按各种优先级定义的优先队列。
- 算法的效率与**广义优先队列**的选择密切相关。
- 如果选用通常的FIFO队列，则在最坏情况下，预流推进算法求最大流所需的计算时间为$O(mn^2)$，其中m和n分别为图G的边数和顶点数。
- 如果以顶点高度值为优先级，选用优先队列实现预流推进算法，则在最坏情况下，求最大流所需的计算时间为 $O(\sqrt{m}n^2)$
- 这个算法也称为最高顶点标号预流推进算法。
- 近来已提出许多其它预流推进算法的实现策略，在最坏情况下算法所需的计算时间已接近$O(mn)$。

最大流问题的变换与应用

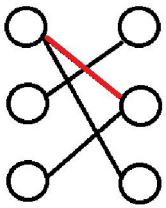


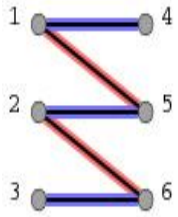
- 1、一般网络的最大流问题：多个源和多个汇
- 2、顶点容量约束：将该顶点u增加一条边(u,v)
- 3、二分图的最大匹配：（P303）

设G=（V，E）是一个图。如果顶点集合V可分割为两个互不相交的子集X，Y，并且图中每条边(i，j)所关联的两个顶点i和j分属于这两个不同的顶点集(i in X,j in Y)，则称图G为一个二分图。

设G=（V，E）是一个图。如果M $\subseteq$ E 且M中任何两条边都不与同一个顶点相关联，则称M是G的一个匹配。G的边数最多的匹配称为G的最大匹配。

二分图的最大匹配问题就是在已知图G的一个二分图的前提下，求G的最大匹配。





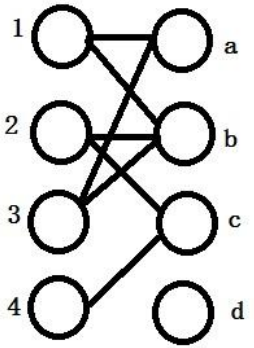
最大流问题的变换与应用

- 如果一个匹配中，图中的每个顶点都和图中某条边相关联，则称此匹配为完全匹配，也称作完备匹配。
- 求二分图最大匹配可以用最大流(Maximal Flow)或者匈牙利算法(Hungarian Algorithm)

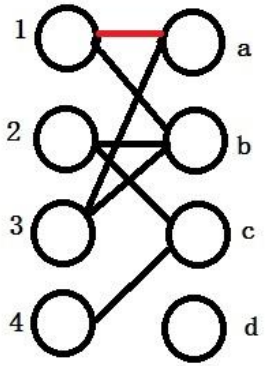
交替路：从一个未匹配点出发，依次经过非匹配边、匹配边、非匹配边...形成的路径叫交替路。
增广路：从一个未匹配点出发，走交替路，如果途径另一个未匹配点（出发的点不算），则这条交替路称为增广路。
注意：把增广路径上的所有第奇数条边加入到原匹配中去，并把增广路径中的所有第偶数条边从原匹配中删除（这个操作称为增广路径的取反），则新的匹配数就比原匹配数增加了1个。

匈牙利算法

- 把左边1，2，3，4和右边a,b,c,d来进行匹配

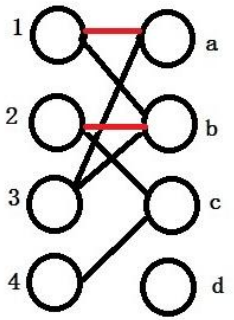


1、给1分配a,1和a之间 连上红线表示建立匹配



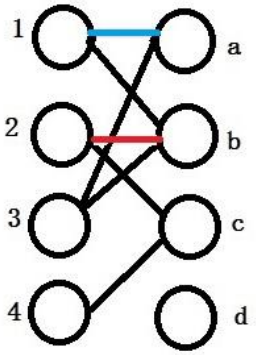
匈牙利算法

2、然后接着给2分配b,2和b连上红线表示匹配。



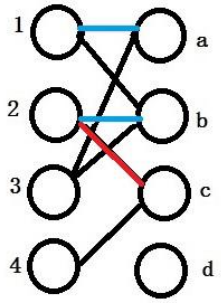
3、紧接着给3分配，这时候发现a,b已经都有所属了；

我们尝试给1重新分配，把原来的分配拆掉，用蓝线表示。

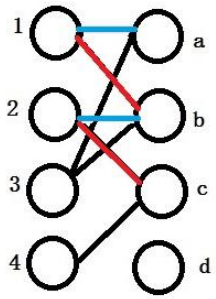


匈牙利算法

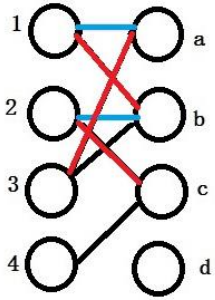
4、但是很快我们发现1重新分配不了，b已经有所属，那么继续尝试给2重新分配，把原来的分配拆掉，用蓝线表示。2重新分配到c,用红线表示。



5、这个时候，1可以重新分配到b,用红线表示。



6、最后，3就可以分配到a,用红线表示。



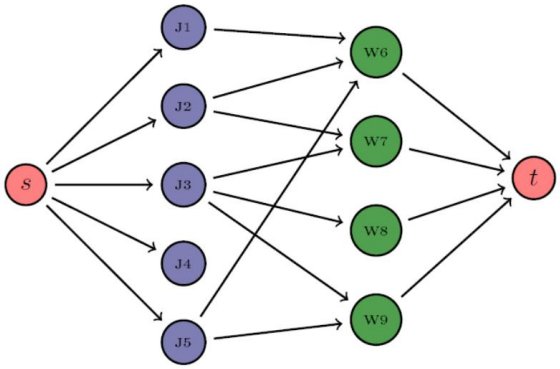
7、对于4，由于c已经被分配，而且尝试给其他1，2，3重新分配无法实现，就此结束。基本原则就是在原有匹配基础上重新分配，看是否可以添加一个新的匹配。

最大流问题的变换与应用

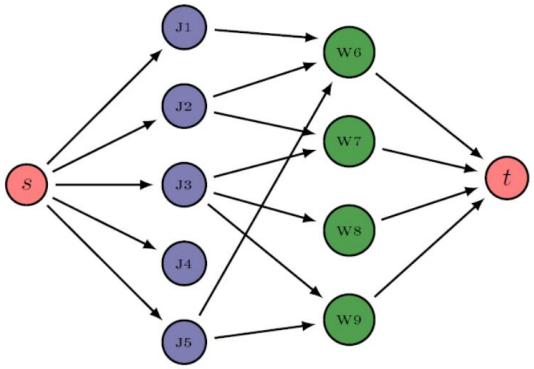
用增广路求解：

开始：加上s和t
容量为1

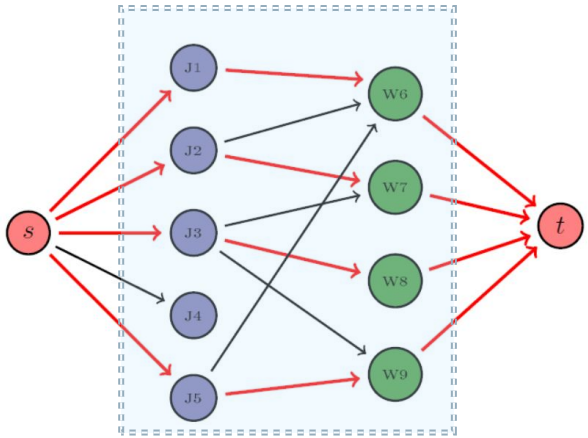
结束：最大匹配



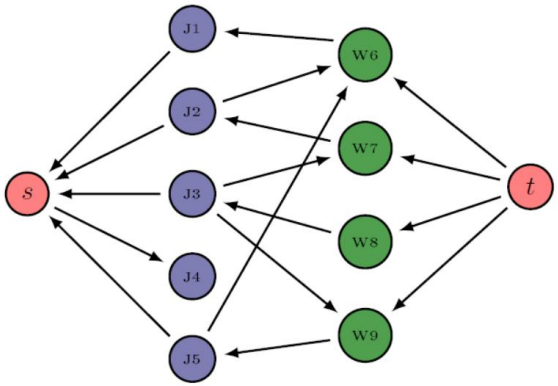
Network Graph



Residual Graph



Add the flow



Residual Graph

8.3 最小费用流问题

- **1 网络流的费用**
- 在实际应用中，与网络流有关的问题，不仅涉及流量，而且还有费用的因素。
- 网络的每一条边(v,w)除了给定容量cap(v,w)外，还定义了一个单位流量费用cost(v,w)。对于网络中一个给定的流flow，其费用定义为：

$$cost(flow) = \sum_{(v,w)\in E} cost(v,w) \times flow(v,w)$$

- **2 最小费用流问题**
- 给定网络G，要求G的一个最大用流flow，使流的总费用最小。

- **3 最小费用可行流问题**
- 给定多源多汇网络G，要求G的一个可行流flow，使可行流的总费用最小。
- 可行流问题等价于最大流问题。最小费用可行流问题也等价于最小费用流问题。

略

8.3 最小费用流问题

- 连续最短路算法（Successive Shortest Path）
- 消圈算法（Cycle Canceling）
- 原始对偶算法（Primal Dual）
- 网络单纯形（Network Simplex）

- 略

略

消圈算法

- **1 算法基本思想**
- 最小费用流问题有关的算法中，仍然沿用残流网络的概念。
- 此时，残流网络中边的费用定义为：
- **int costRto(int v) { return from(v) ? -pcost : pcost; }**
- 当残流网络中的边是向前边时，其费用不变。
- 当残流网络中的边是向后边时，其费用为原费用的负值。
- 由于残流网络中存在负费用边，因此残流网络中就不可避免地会产生负费用圈。
- 在与最小费用流问题有关的算法中，负费用圈是一个重要概念。

- **最小费用流问题的最优性条件**
- 网络G的最大流flow是G的一个最小费用流的充分且必要条件是flow所相应的残流网络中没有负费用圈。

略

- **最小费用流的消圈算法**

步骤0：用最大流算法构造最大流flow。

步骤1：如果残量网络中不存在负费用圈，则计算结束，已经找到最小费用流；

否则转步骤2。

步骤2：沿找到的负费用圈增流，并转步骤1。

略

- **3 算法的计算复杂性**
- 给定网络中有n个顶点和m条边，且每条边的容量不超过M，每条边的费用不超过C。
- 最大流的费用不超过mCM，而每次消去负费用圈至少使得费用下降1个单位，因此最多执行mCM次找负费用圈和增流运算。
- 用Bellman-Ford算法找1次负费用圈需要O(mn)计算时间。
- 最小费用流的消圈算法在最坏情况下需要计算时间 $O(m^2nCM)$

略

最小费用路算法

- **1 算法基本思想**
- 消圈算法首先找到网络中的一个最大流，然后通过消去负费用圈使费用降低。
- 最小费用路算法不用先找最大流，而是用类似于求最大流的增广路算法的思想，不断在残流网络中寻找从源s到汇t的最小费用路，然后沿最小费用路增流，直至找到最小费用流。
- 残流网络中从源s到汇t的最小费用路是残流网络中从s到t的以费用为权的最短路。
- 残流网络中边的费用定义为：

$$wt(v,w)=\begin{cases}\cos t(v,w) & (v,w)\in P^{+}\\ -\cos t(w,v) & (v,w)\in P^{-}\end{cases}$$

- 当残流网络中边(v,w)是向前边时，其费用为cost(v,w)；
- 当(v,w)是向后边时，其费用为-cost(w,v)。

略

最小费用流的最小费用路算法

步骤0：初始可行0流。

步骤1：如果不存在最小费用路，则计算结束，已经找到最小费用流；

否则用最短路算法在残流网络中找从s到t的最小费用可增广路，转步骤2。

步骤2：沿找到的最小费用可增广路增流，并转步骤1。

略

- **3 算法的计算复杂性**
- 算法的主要计算量在于连续寻找最小费用路并增流。
- 给定网络中有n个顶点和m条边，且每条边的容量不超过M，每条边的费用不超过C。
- 每次增流至少使得流值增加1个单位，因此最多执行M次找最小费用路算法。
- 如果找1次最小费用路需要s(m,n,C)计算时间，则求最小费用流的最小费用路算法需要O(Ms(m,n,C))计算时间。

略

网络单纯形算法

- **1 算法基本思想**
- 消圈算法的计算复杂度不仅与算法找到的负费用圈有关，而且与每次找负费用圈所需的时间有关。
- 网络单纯形算法是从解线性规划问题的单纯形算法演变而来，但从算法的运行机制来看，可以将网络单纯形算法看作另一类消圈算法。
- 其基本思想是用一个可行支撑树结构来加速找负费用圈的过程。
- 对于给定的网络G和一个可行流，相应的**可行支撑树**定义为G的一棵包含所有弱流边的支撑树。
- 网络单纯形算法的第一步是构造可行支撑树。
- 从一个可行流出发，不断找由弱流边组成的圈，然后沿找到的弱流圈增流，消除所有弱流圈。
- 在剩下的所有弱流边中加入零流边或饱和边构成一棵可行支撑树。
- 在可行支撑树结构的基础上，网络单纯形算法通过顶点的势函数，巧妙地选择非树边，使它与可行支撑树中的边构成负费用圈。然后，沿找到的负费用圈增流。

略

- 定义了顶点的势函数Φ后，残流网络中各边(v,w)的势费用定义为：
- c*(v,w)=c(v,w)-(Φ(v)- Φ(w))。
- 其中，c(v,w)是(v,w)在残流网络中的费用。
- 如果对可行支撑树中所有边（v,w）有c*(v,w)=0，则相应的势函数Φ是一个有效势函数。
- 对于一棵可行支撑树，如果将一条非树边加入可行支撑树，产生残流网络中的一个负费用圈，则称该非树边为一条可用边。

略

- **可用边定理：**给定一棵可行支撑树及其上的一个有效势函数，非树边e是一条可用边的充分必要条件是，e是一条有正势费用的饱和边，或e是一条有负势费用的零流边。
- 事实上，设e=(v,w)。
- 边e与树边t1,t2,...,td构成一个圈cycle: t1,t2,...,t_d，t1, 其中v=t1，w=t_d 。
- 按照边的势费用的定义有：
- $c(w,v)=c^*(w,v)+\Phi(t_d)-\Phi(t_1)$
- $c(t1,t2)=\Phi(t1)-\Phi(t2)$
- $c(t2,t3)=\Phi(t2)-\Phi(t3)$
- ...
- $c(t_{d-1}, t_d)=\Phi(t_{d-1})-\Phi(t_d)$

略

- 各式相加得：cost(cycle)=c*(w,v)。
- 由此可见，e是一条可用边当且仅当cost(cycle)<0；
- 当且仅当c*(w,v)<0；
- 当且仅当e是一条有正势费用的饱和边或e是一条有负势费用的零流边。
- **最优性条件：**给定网络G的可行流flow及相应的可行支撑树T，如果不存在T的可用边，则flow是一个最小费用流。
- 事实上，如果不存在T的可用边，则由可用边的定义知残流网络中没有负费用圈。又由最小费用流问题的最优性条件知flow是一个最小费用流。

略

最小费用流的网络单纯形算法

步骤0：构造flow为初始可行0流。

构造相应的可行支撑树T和有效的顶点势函数。

步骤1：如果不存在T的可用边，则计算结束，已经找到最小费用流；否则转步骤2。

步骤2：选取T的一条可用边与T的树边构成负费用圈，沿找到的负费用圈增流，

从T中删去一条饱和边或零流边，重构可行支撑树，并转步骤1。

略

- **3 算法的计算复杂性**
- 给定网络中有n个顶点和m条边，且每条边的容量不超过M，每条边的费用不超过C。
- 最大流的费用不超过mCM，而每次消去负费用圈至少使得费用下降1个单位，因此最多执行mCM次找负费用圈和增流运算。
- 用网络单纯形算法找1次负费用圈需要O(m)计算时间。
- 因此，求最小费用流的网络单纯形算法在最坏情况下需要计算时间 $O(m^2CM)$

略